Availability Group Login Synchronization || Onprem SQL Server

Automated dbatools Implementation SOP

Document Version:
1.0 | Date: 21stJuly 2026 | Audience: SQL Server DBA, Infrastructure Team || Drafted by Praveen Madupu

Executive Summary

This SOP provides a repeatable, automated process for keeping SQL Server Availability Group logins synchronized across all replica instances. Manual login synchronization is error-prone and labor-intensive. This procedure leverages the dbatools PowerShell module to eliminate manual work and ensure consistency across the entire Availability Group estate.

Problem Statement

Managing SQL Server Availability Groups introduces operational complexity around login synchronization:

- New logins added to primary are not automatically propagated to secondaries
- Manual login creation on each secondary is repetitive, time-consuming, and error-prone
- In multi-replica environments, coordination becomes increasingly difficult
- Login updates must be applied separately to each secondary replica
- Failover scenarios may expose gaps in login parity, impacting applications

Solution Overview

This SOP automates login synchronization using a PowerShell script powered by dbatools. The script:

- Connects to the Availability Group listener (single entry point)
- Automatically detects the current primary and all secondary replicas
- Queries all logins from the primary replica
- Applies those logins to each secondary replica
- Can be scheduled to run on a recurring basis
- Provides detailed logging for audit and troubleshooting purposes

Prerequisites

Before implementing this procedure, ensure the following are in place:

- dbatools PowerShell module version 1.0.0 or later
- PowerShell 5.0 or later on the execution host
- SQL Server 2012 or later supporting Availability Groups
- One or more Availability Groups already configured and operational
- Network connectivity from the execution host to all AG replica instances
- SQL account with sysadmin privileges on all primary and secondary replicas
- Windows Task Scheduler or SQL Agent for automation

Installation and Setup

1. Install dbatools Module

Run the following command in an elevated PowerShell prompt:

```
Install-Module dbatools -Force -SkipPublisherCheck
```

2. Create Script Directory

Create: C:\SQLAdmin\AGSyncLogins\

3. Create Log Directory

Create: C:\Logs\AGSyncLogins\

Script Implementation

The PowerShell script performs the following operations:

- Connects to the Availability Group listener
- Retrieves all replica instances using Get-DbaAgReplica cmdlet
- Identifies the primary and all secondary replicas
- Copies all logins to each secondary using Copy-DbaLogin
- Logs all operations with timestamps and severity levels
- Handles errors gracefully with exception reporting

Key Script Parameters:

- AvailabilityGroupListener: Name of the AG listener (required)
- LogPath: Directory for log files (default: C:\Logs\AGSyncLogins\)
- ClientName: Connection identifier (default: AG Login Sync helper)
- WhatIf: Preview changes without applying them (recommended for testing)

Execution Instructions

1. Dry-Run Execution (Initial Test)

Before scheduling automation, execute the script in WhatIf mode to preview changes:

```
.\SyncLoginsToReplica.ps1 -AvailabilityGroupListener "AGListenerName" -WhatIf
```

Review the output to confirm:

- Primary replica is correctly identified
- All secondary replicas are listed
- Login copy operations preview correctly
- No errors or unexpected warnings

2. Live Execution

Once validated, run without the -WhatIf switch to execute actual changes:

```
.\SyncLoginsToReplica.ps1 -AvailabilityGroupListener "AGListenerName"
```

Scheduling Automation

Windows Task Scheduler (Recommended)

Step 1: Create Scheduled Task

- Name: Sync AG Logins - AGListenerName
- Trigger: Hourly (or every 10-15 minutes for high-volatility environments)
- Action: Start a Program
- Program: powershell.exe

Arguments:

-NoProfile -ExecutionPolicy Bypass -File "C:\SQLAdmin\AGSyncLogins\SyncLoginsToReplica.ps1" -AvailabilityGroupListener "AGListenerName"

Step 2: Configure Run Conditions

- Run with highest privileges: Enabled
- Run whether user is logged in: Enabled
- Account: Dedicated service account with sysadmin rights on all replicas

Monitoring and Logging

Log File Locations

All executions write to: C:\Logs\AGSyncLogins\SyncLoginsToReplica_YYYYMMDD_HHMMSS.log

Log Entry Format

- [INFO]: Successful operations and status messages
- [WARN]: Non-fatal issues requiring attention
- [ERROR]: Critical failures requiring immediate investigation

Proactive Monitoring

- Set up alerts for ERROR entries in log files
- Monitor scheduled task failures in Windows Event Log
- Flag missing log files as script failure indicators
- Track failure rates across multiple executions

Troubleshooting

No primary replica found:

- Verify listener name.
- Confirm AG is healthy and primary is online.
- Check network connectivity to listener.

Connection denied to secondary:

- Verify account has sysadmin rights.
- Check firewall rules.
- Confirm SQL Server running on secondary.

dbatools module not found: Run Install-Module dbatools -Force.

Set execution policy: Set-ExecutionPolicy RemoteSigned

Login already exists:

- Script handles gracefully by default.
- Verify no conflicts. Manually drop/recreate if needed.

Scheduled task not running:

- Check Event Log for errors.
- Verify batch logon rights.
- Test manual execution.

Best Practices

- Execution Frequency: Run hourly for most environments
- Stagger multiple AGs to avoid network congestion
- Always test with -WhatIf on new AGs before live execution
- Use dedicated service account with minimal required privileges
- Archive logs older than 90 days; maintain 30 days online for troubleshooting
- Include AG failover scenarios in change management
- Keep scripts in version control with change logs
- Document AG listener names, schedule, and responsible team

References

- dbatools Documentation: https://dbatools.io/
- dbatools GitHub: https://github.com/dataplat/dbatools
- Slack Community: https://dbatools.io/slack
- Original Article: https://dbatools.io/keeping-availability-group-logins-in-sync-automatically/
- SQL Server Availability Groups: https://docs.microsoft.com/sql/database-engine/availability-groups/

Version: 1.0 | Date: July 2026 | Source: dbatools community (Andreas Schubert, 2019-2025)

Source: https://dbatools.io/keeping-availability-group-logins-in-sync-automatically/

Availability Group Login Synchronization

Automated dbatools PowerShell Solution

The Problem

- Manual login sync required on each secondary replica
- Human errors and missed login updates
- Login parity gaps after failover impact applications

The Automated Solution

Step 1
Install dbatools
PowerShell Module

Step 2
Connect to AG Listener
Discover All Replicas

Step 3
Copy Logins to All
Secondary Replicas

How It Works

AG Listener (Single Entry Point)

Primary Replica
Reads all SQL Server logins and syncs to every secondary

Secondary Replica 1
Logins Applied

Secondary Replica 2
Logins Applied

Secondary Replica N
Logins Applied

Scheduling & Automation

Run Automatically
Hourly Schedule (configurable)
Windows Task Scheduler or SQL Agent
Dedicated service account

Monitor & Alert
Log files track all operations
[INFO] Success [WARN] Issues [ERROR] Failures
Set up alerts for error entries

Key Benefits

✓ Zero Manual Work
Fully automated sync every hour

✓ Login Parity
All replicas always in sync

✓ Reliable Failover
No access issues post-failover

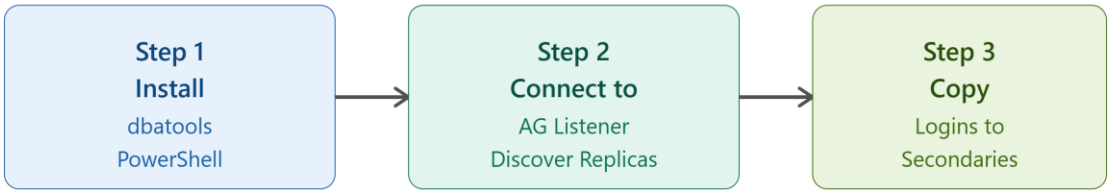
Availability Group Login Synchronization

Automated dbatools PowerShell Solution

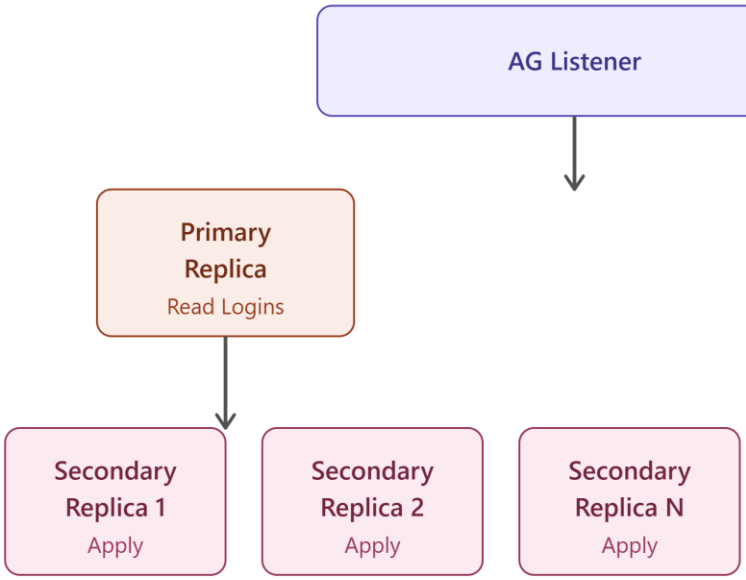
The Problem

- ❌ Manual login sync on each secondary
- ❌ Human errors and missed updates
- ❌ Login parity gaps after failover

The Automated Solution



How It Works



Scheduling & Monitoring

⏰ Automation

Run hourly via:

- Windows Task Scheduler
- SQL Server Agent

📊 Monitoring

Log files track:

- ✓ [INFO] Success
- ✓ [WARN] Issues

Key Benefits

✓ Zero Manual Work
Fully automated

✓ Login Parity
All replicas in sync

✓ Reliable Failover
No access issues